Movie Recommender System Using Machine Learning

A project report submitted in partial fulfillment of the requirements for the award of degree in

MASTER OF COMPUTER APPLICATIONS

Submitted By

Adigarla Sai Dhanya Aasritha

Regd. No: 2024021839

Under the esteemed guidance of

Dr. Sai Shaktimayee Sahu

Assistant Professor

GITAM

DEEMED TO BE UNIVERSITY

DEPARTMENTOF COMPUTER SCIENCE

GITAM SCHOOL OF SCIENCE

GITAM(Deemed to be University)

VISAKHAPATNAM- 530045, AP

(2025-26)

Movie Recommender System Using Machine Learning

A project report submitted in partial fulfilment of the requirements for the award of degree in

Master of Computer Applications

By

Adigarla Sai Dhanya Aasritha

Regd. No: 2024021839

Under the esteemed guidance of

Dr. Sai Shaktimayee Sahu

Assistant Professor

Department of Computer Science

GITAM School of Science

GITAM (Deemed to be University)

Visakhapatnam -530045, A.P

(2025-2026)

CERTIFICATE

This is to certify that the project entitled "**Movie Recommender System Using Machine Learning**" is a Bonafide work done by **Adigarla Sai Dhanya Aasritha**, Regd. No: 2024021839 during December 2025 to April 2026 in partial fulfillment of the requirement for the award of degree of Master of Computer Applications in the Department of Computer Science, GITAM School of Science, GITAM (Deemed to be University), Visakhapatnam.

Internal Guide:	**Head of the Department:**
Dr. Sai Shaktimayee Sahu	**Dr. Uma Devi T**
Assistant Professor	**Professor**
Dept. of Computer Science	**Dept. of Computer Science**
GITAM School of Science	**GITAM School of Science**

DECLARATION

I **Adigarla Sai Dhanya Aasritha,** Regd. No: 2024021839 hereby declare that the project entitled "**Movie Recommender System Using Machine Learning"** is an original work done in the partial fulfillment of the requirements for the award of degree of Master of Computer Applications in GITAM School of Science, GITAM (Deemed to be University), Visakhapatnam. I assure that this project work has not been submitted towards any other degree or diploma in any other colleges or universities.

Adigarla Sai Dhanya Aasritha
(2024021839)

ACKNOWLEDGEMENT

I hereby take the opportunity to express my sincere gratitude to the following eminent personalities whose advice and timely suggestions helped me to complete this project work successfully without any difficulty.

I would like to express my sincere gratitude to our honourable Principal and Prof. S. Anantha Ramakrishna , GITAM School of Science, GITAM (Deemed to be University), for giving me an opportunity to work on this project.

I would like to thank Prof. T. UMA DEVI, Head of the Department, Department of Computer Science, GITAM School of Science, GITAM (Deemed to be University), for giving me an opportunity to work on this project.
I would like to thank my guide Dr. Sai Shaktimayee Sahu, Assistant Professor, Department of Computer Science, GITAM School of Science, GITAM (Deemed to be University) for her valuable guidance and support for the completion of my project.

I would like to thank my coordinators Dr. Sri Venkatesh Muktevi , Dr. Nallamalla Anusha, and Dr. M Sesha Shayee Assistant Professors, Department of Computer Science, GITAM School of Science, GITAM (Deemed to be University) for his valuable support for the completion of my project.

My special regards to all the faculty members and supporting staff in the Department of Computer Science, GITAM School of Science, GITAM (Deemed to be University) who have helped me in the completion of my project.

Adigarla Sai Dhanya Aasritha

(2024021839)

ABSTRACT

The rapid expansion of online streaming platforms and digital movie libraries has made it difficult for users to select movies that match their interests. Traditional movie selection methods mainly rely on popularity, ratings, or manual search, which are often time-consuming and fail to reflect individual user preferences. To overcome this challenge, this project presents a Movie Recommendation System that provides personalized movie suggestions using data science and machine learning techniques.

The proposed system analyses user behaviour such as viewing history, ratings, and preferences along with movie attributes like genre, cast, director, and release year. It employs both content-based filtering and collaborative filtering techniques to generate accurate and meaningful recommendations. Content-based filtering suggests movies similar to those previously liked by the user, while collaborative filtering recommends movies preferred by users with similar tastes. A hybrid approach is used to combine the strengths of both methods.

By identifying similarity patterns between users and movies, the system delivers personalized recommendations that reduce search effort and improve user satisfaction. The implementation of this system demonstrates the practical application of data science concepts in real-world recommendation systems and enhances user engagement on online movie platforms.

INDEX

TABLE OF CONTENTS		PG.NO
1.	**CHAPTER 1: INTRODUCTION**	**1**
1.1	Background	2
1.2	Motivation	2
1.3	Importance of Data Science in the Domain	2
1.4	Problem Statement	3
1.5	Objectives of the Project	3
1.6	Scope of the Project	3
1.7	Organization of the Report	4
2.	**CHAPTER 2: LITERATURE REVIEW**	**5**
2.1	Review of Existing Systems	6
2.2	Comparative Summary of Existing Approaches	7
2.3	Research Gaps Identified	9
2.4	Justification for the Proposed Work	9-10
3.	**CHAPTER 3: DATASET DESCRIPTION**	**11**
3.1	Source of Dataset	11
3.2	Dataset Size (Records & Attributes)	11
3.3	Attribute Description Table	12
3.4	Data Types	12-13

3.5	Data Quality Issues	14
4.	**CHAPTER 4: METHODOLOGY**	**15**
4.1	System Architecture	15-19
4.2	Data Preprocessing	19-22
4.3	Model Design	22-25
4.4	Tools & Technologies	25-26
5.	**CHAPTER 5: IMPLEMENTATION**	**27**
5.1	Modular wise implementation	27-28
5.2	Pseudocode/Algorithm steps	28
5.3	Code snippets	29-53
5.4	Screenshots of Outputs	54-60
6.	**CHAPTER 6: RESULTS AND PERFORMANCE ANALYSIS**	**61**
6.1	Output Samples	61
6.2	Performance metrices	61-62
6.3	Confusion Matrix	63
6.4	Graphical Analysis	63
6.5	Comparison with Baseline Methods	63
7.	**CHAPTER 7: DISCUSSION**	**64**
7.1	Interpretation of Results	64
7.2	Strengths of the Model	64

7.3	Limitations of the Project	64
7.4	Observations	65
8	**CHAPTER 8: CONCLUSION AND FUTURE SCOPE**	66
8.1	Summary of Work	66
8.2	Key Contributions	66
8.3	Learning Outcomes	67
8.4	Possible extensions	67
	References	68

CHAPTER 1: INTRODUCTION

This project focuses on developing a Movie Recommendation System, which is designed to suggest movies to users based on their preferences, interests, and past behavior. With the rapid growth of digital platforms and online streaming services, users are exposed to a vast collection of movies. While this offers more choices, it also creates difficulty in selecting the right content. To solve this problem, recommendation systems play an important role by filtering relevant content and providing personalized suggestions.

The system uses machine learning techniques to analyze user data such as ratings, watch history, and preferences. It identifies patterns in user behavior and recommends movies that are most likely to match the user's taste. The system mainly uses techniques such as content-based filtering, collaborative filtering, or a hybrid approach that combines both methods for better accuracy.

In content-based filtering, the system recommends movies similar to those the user has liked before by analyzing features like genre, cast, keywords, and storyline. In collaborative filtering, recommendations are made based on the behavior of similar users. For example, if two users have similar tastes, the system suggests movies liked by one user to the other.

The dataset used in this project includes information such as movie titles, genres, user ratings, and other metadata. Before applying machine learning algorithms, the data is cleaned and preprocessed to remove missing values and inconsistencies. After preprocessing, similarity measures such as cosine similarity or correlation are used to find relationships between movies or users.

A user-friendly interface is developed where users can input their preferences or select a movie they like. Based on this input, the system generates a list of recommended movies along with similarity scores or ranking. This makes the system easy to use even for non-technical users.

Overall, this project demonstrates how data science and machine learning can be applied in real-world applications to improve user experience. It replicates the working principles of popular platforms like Netflix and Amazon Prime by delivering personalized recommendations efficiently.

1.1 Background of the Problem

In today's digital era, streaming platforms provide access to a huge number of movies across different genres, languages, and regions. While this increases user choice, it also leads to a problem known as information overload. Users often find it difficult to choose a movie because of the large number of options available.

Most traditional systems recommend movies based on overall ratings, trending lists, or popularity. However, these methods do not consider individual user preferences. For example, a user who likes action movies may still receive recommendations for romantic or comedy movies based on popularity. This mismatch reduces user satisfaction.

Additionally, users spend a significant amount of time searching for movies instead of watching them. This highlights the need for an intelligent system that can understand user preferences and provide accurate recommendations quickly.

1.2 Motivation for the Project

The main motivation behind this project is to improve the movie selection experience for users. A well-designed recommendation system can save time and effort by suggesting relevant movies instantly.

This project also provides an opportunity to understand and apply important concepts of data science and machine learning, such as:

- Data preprocessing
- Feature extraction
- Similarity measurement
- Model building

By working on this project, we gain practical knowledge of how real-world recommendation systems function and how they can be improved.

1.3 Importance of Data Science in the Chosen Domain

Data science plays a crucial role in building recommendation systems. It helps in analyzing large amounts of data and extracting meaningful patterns. In the entertainment industry, user interaction data such as ratings, clicks, and watch time is used to understand user behavior.

Machine learning algorithms enable systems to learn from this data and improve recommendations over time. As more data is collected, the system becomes more accurate

and personalized. This not only enhances user satisfaction but also increases user engagement and retention for streaming platforms.

1.4 Problem Statement

Existing movie recommendation systems mainly provide generalized suggestions based on popularity or average ratings. These systems fail to consider individual user preferences, similarity between users, and changing interests over time.

As a result:

- Recommendations are often repetitive
- Users may not find relevant movies
- Personalization is limited

Therefore, there is a need to develop a system that provides accurate, personalized, and dynamic movie recommendations.

1.5 Objectives of the Project

The main objectives of this project are:

- To design and develop an intelligent movie recommendation system
- To analyze user behavior using ratings and viewing history
- To implement content-based and collaborative filtering techniques
- To improve the accuracy and personalization of recommendations
- To reduce the time spent by users in searching for movies
- To create a simple and user-friendly interface

1.6 Scope of the Project

The scope of this project is focused on recommending movies using structured datasets such as MovieLens. The system is limited to analyzing available features like genres, ratings, and user interactions.

However, the system can be extended in the future by:

- Adding sentiment analysis from user reviews
- Including deep learning models for better predictions
- Recommending other content such as web series, music, and books
- Integrating with real-time streaming platforms

1.7 Organization of the report

The report is organized in a structured manner to provide a clear understanding of the development and implementation of the Movie Recommendation System. It allows the reader to follow the project from problem definition to final results without requiring prior knowledge of the system.

The report begins with an introduction that explains the background of recommendation systems, the need for personalized movie suggestions, and the problem statement. It also outlines the objectives and scope of the project.

The literature review presents an overview of existing recommendation techniques such as content-based filtering, collaborative filtering, and hybrid models. It also highlights the limitations of traditional systems and the need for improved approaches.

The dataset description section provides details about the dataset used, including its source, attributes, and data quality issues. It also explains how the data is prepared for building the recommendation system.

The methodology chapter describes the system architecture, data preprocessing techniques, and model design. It explains how content-based filtering using TF-IDF and cosine similarity, collaborative filtering using SVD, and the hybrid recommendation approach are implemented.

The implementation chapter explains the technologies used, system modules, and workflow of the application. It also describes how the user interface is developed and how recommendations are generated in real time.

The results and performance analysis section presents sample outputs, evaluation metrics such as accuracy, precision, recall, RMSE, and graphical analysis. It also compares the proposed system with traditional recommendation methods.

The discussion section highlights the interpretation of results, strengths, limitations, and observations of the system.

Finally, the conclusion and future scope summarize the overall work, key contributions, and possible improvements that can be made in the future.

CHAPTER 2: LITERATURE REVIEW

Movie recommendation systems have been widely studied as an effective solution to help users discover relevant content from large collections of movies. With the rapid growth of digital platforms and online streaming services, users now have access to thousands of movies across different genres and languages. While this provides more choices, it also creates confusion and difficulty in selecting suitable content. Recommendation systems solve this problem by automatically suggesting movies based on user preferences, thereby improving user experience and saving time.

Traditional recommendation approaches mainly include content-based filtering and collaborative filtering techniques. Content-based filtering recommends movies based on similarities in features such as genre, cast, keywords, and descriptions. It analyzes the movies that a user has previously liked and suggests similar ones. This method is simple and provides personalized recommendations, but it may lack diversity since it tends to recommend similar types of movies repeatedly.

Collaborative filtering focuses on user behavior instead of movie features. It uses information such as user ratings, watch history, and preferences to identify users with similar interests. Based on this, the system recommends movies that other similar users have liked. This approach is effective in providing diverse recommendations but faces challenges such as the cold-start problem, where new users or new movies have insufficient data for making recommendations.

Recent research in recommendation systems has focused on hybrid approaches that combine both content-based and collaborative filtering techniques. Hybrid systems aim to overcome the limitations of individual methods by using both movie features and user behavior. This results in more accurate, diverse, and personalized recommendations.

Machine learning techniques play an important role in modern recommendation systems. Algorithms such as cosine similarity, matrix factorization, clustering, and deep learning are used to identify patterns in user preferences and movie data. These techniques help the system learn from past interactions and improve recommendations over time.

Recommendation systems depend heavily on data such as user ratings, movie metadata, and interaction history. The quality of this data directly affects the performance of the system.

Therefore, proper data preprocessing, cleaning, and feature engineering are essential steps in building an effective recommendation system.

This project is based on these existing approaches and implements a hybrid recommendation system that combines content-based and collaborative filtering techniques. It uses methods such as TF-IDF and cosine similarity for content-based filtering and SVD for collaborative filtering. The system aims to provide accurate, personalized, and efficient movie recommendations through a simple and user-friendly interface**.**

2.1 Review of Existing Systems / Studies

Traditional movie recommendation methods mainly rely on manual searching or browsing through categories. Users typically select movies based on genre, popularity, ratings, or trending lists. While these methods are simple, they do not consider individual user preferences. As a result, users may receive general recommendations that are not aligned with their interests, leading to lower satisfaction and increased time spent searching for suitable content.

Content-based recommendation systems were introduced to improve personalization. These systems recommend movies similar to those that a user has liked in the past by analyzing features such as genre, keywords, cast, director, and storyline. For example, if a user watches action movies frequently, the system will suggest other action movies. This approach is simple, effective, and does not require data from other users. However, one major limitation is that it may lack diversity, as it tends to recommend movies with similar characteristics, limiting exploration of new content.

Collaborative filtering systems take a different approach by focusing on user behavior rather than movie features. These systems analyze user interactions such as ratings, watch history, and preferences to identify users with similar tastes. Based on this similarity, the system recommends movies liked by other users with similar interests. Collaborative filtering is widely used because it can provide more diverse and accurate recommendations. However, it suffers from challenges such as the cold-start problem, where new users or new movies have insufficient data, and data sparsity, where user interaction data is limited.

Modern streaming platforms such as Netflix and Amazon Prime Video use advanced hybrid recommendation systems. These systems combine both content-based and collaborative filtering techniques to improve recommendation accuracy and diversity. They also use advanced machine learning and deep learning models to analyze user behavior, watch time, and interaction patterns. Hybrid systems overcome many limitations of individual approaches but require large datasets, high computational power, and complex infrastructure to function effectively.

Some existing research systems also incorporate additional techniques such as clustering, sentiment analysis, and deep learning models to further enhance recommendation quality. These systems aim to capture more complex user preferences and provide dynamic recommendations that adapt over time.

In comparison to these complex systems, this project focuses on a lightweight and efficient hybrid recommendation approach that combines content-based and collaborative filtering techniques. The system is designed to be simple, user-friendly, and effective in providing relevant movie suggestions based on user input. It balances performance and simplicity, making it suitable for academic implementation while still demonstrating the core concepts of modern recommendation systems.

Modern recommendation systems use advanced machine learning techniques to provide accurate and personalized suggestions. They combine content-based filtering and collaborative filtering in hybrid models to improve performance. These systems analyze user behavior such as ratings, watch history, and interaction patterns to understand preferences. Techniques like matrix factorization (SVD), cosine similarity, and deep learning are used to identify hidden patterns in data. They also handle challenges like cold-start and data sparsity using intelligent algorithms. Real-time data processing and continuous learning help improve recommendations over time. Overall, modern systems focus on personalization, scalability, and efficiency to enhance user experience.

2.2 Comparative Summary of Existing Techniques

Title	Author(s)	Description	Methodology	Drawbacks	Results
Hybrid Movie Recommendation System (2025)	Sahu, S., Kumar, R., et al.	A multi-modal recommendation system combining deep learning with traditional machine learning approaches.	RoBERTa embeddings for textual data, XGBoost for rating prediction, and SVD++ for collaborative filtering.	High computational cost and sensitivity to data sparsity.	Improved accuracy by **0.59% to 4.6%** compared to traditional SVD models.
A Personalized Movie Recommendation System Based on Improved NCF (2025)	Research Team (Applied & Computational Engineering)	Uses deep learning techniques to capture complex, non-linear user–movie interactions.	Neural Collaborative Filtering (NCF) using embedding layers and Multi-Layer Perceptrons (MLP).	Performance declines when dataset dimensions or metadata are limited.	Achieved a high **Hit Ratio@10**, outperforming linear dot-product models.
Movie Recommendation in Exact Prediction using GOA-K-Means (2025)	Sahu, S., et al.	Focuses on optimizing clustering to better model evolving user interests.	Grasshopper Optimization Algorithm (GOA) integrated with K-Means clustering and PCA.	Increased complexity in real-time calibration for large-scale datasets.	Significant reduction in **MAE and RMSE** on the MovieLens-1M dataset.
A Research Paper on Movie Recommendation Systems (2025)	IJARCCE Editorial Team	Benchmark study comparing single-model and hybrid recommendation approaches.	Hybrid model combining TF-IDF (content-based) and collaborative filtering using weighted scoring.	Collaborative filtering continues to face cold-start issues for new users.	Hybrid model achieved **RMSE = 0.86** with the highest precision and recall.

2.3Research Gaps Identified

• Lack of accurate personalization in traditional systems

• Cold-start problem for new users and new movies

• Dependence on a single recommendation technique

• Limited adaptability to changing user preferences

• Data sparsity due to insufficient user interaction data

• Difficulty in handling large-scale datasets efficiently

• Limited diversity in recommendations (repetitive suggestions)

• Lack of real-time recommendation updates

• Inability to capture complex user behavior patterns

• High computational cost in advanced recommendation models

• Poor handling of noisy and incomplete data

• Limited integration of contextual factors such as mood or time

2.4 Justification for the Proposed Work

Lack of accurate personalization in traditional systems – Most systems provide general recommendations based on popularity rather than individual user preferences.

• Cold-start problem for new users and new movies – When there is no prior data available, it becomes difficult for the system to generate meaningful recommendations.

• Dependence on a single recommendation technique – Systems using only content-based or collaborative filtering often fail to provide balanced and accurate results.

• Limited adaptability to changing user preferences – User interests may change over time, but many systems do not update recommendations dynamically.

• Data sparsity due to insufficient user interaction data – Many users provide very few ratings, making it difficult to identify patterns.

• Limited diversity in recommendations – Systems may repeatedly suggest similar types of movies, reducing user exploration.

• Difficulty in handling large-scale datasets – As data grows, managing and

processing it efficiently becomes challenging.

• Lack of real-time recommendation updates – Many systems do not update suggestions instantly based on recent user activity.

• Inability to capture complex user behavior patterns – Basic models fail to understand deeper user preferences and hidden patterns.

• High computational cost in advanced models – Complex algorithms require more time and resources, making them difficult to implement.

• Poor handling of noisy or incomplete data – Missing or inconsistent data can affect recommendation quality.

• Limited use of contextual information – Factors like user mood, time, or location are often ignored.

CHAPTER 3: DATASET DESCRIPTION

3.1 Source of Dataset

The dataset used in this project is obtained from publicly available movie datasets such as Kaggle and TMDB (The Movie Database) and other open movie repositories. These sources provide detailed information about movies including titles, genres, ratings, and user interactions.

Additionally, some data is collected from publicly accessible APIs and datasets to ensure a wide variety of movies and updated information. These sources help maintain accuracy and diversity in recommendations.

Using reliable and publicly available datasets ensures that the system generates accurate and meaningful movie recommendations.

3.2 Dataset Size (Records & Attributes)

The dataset consists of approximately **10,000+ movies** and **100,000+ user ratings**, which are used for generating recommendations.

Each movie is treated as a record, and user ratings are used to understand user preferences.

The dataset includes the following key attributes:

- Movie ID
- Title
- Genres
- Ratings
- User ID
- Timestamp

These attributes help in building both content-based and collaborative filtering models.

3.3 Attribute Table

Attribute Name	Data Type	Description
User_Id	Integer	Unique ID assigned to each user
Movie_Id	Integer	Unique ID assigned to each movie
Title	Text	Name of the movie
Genres	Text	Categories like Action, Comedy, Drama
Rating	Float	User rating (1–5 scale)
Timestamp	DateTime	Time when rating was given

3.4 Data Types Used in the Project

The project uses multiple types of data to build an effective movie recommendation system:

- **Numerical Data:** Includes user ratings, movie IDs, and user IDs, which are used for collaborative filtering and prediction tasks.
- **Categorical Data:** Includes movie genres and user preferences, which help in grouping and filtering movies.
- **Text Data:** Includes movie titles, keywords, cast information, and descriptions, which are processed using TF-IDF for content-based filtering.
- **Metadata:** Includes additional information such as movie categories and user profile details, which support personalization.

Image and time-series data are not used in this project, as the system focuse structured and textual data for generating recommendations.

This combination of data types helps in improving the accuracy and personalization of the recommendation system.

	A	B	C	D	E	F	G	H	I	J	K
1	vie_id	tmdb_id	title	genres	keywords	cast	overview	poster_pa	runtime	vote_average	
2	1	862	Toy Story (	['Family', '(	['rescue', '	['Tom Han	Led by Wo	/uXDfjJbdI	81	7.97	
3	2	8844	Jumanji (1	['Adventur	['giant inse	['Robin Wi	When sibli	/vgpXmVa	104	7.245	
4	3	15602	Grumpier	['Romance	['fishing', '	['Walter M	A family w	/1FSXpj5e8	101	6.483	
5	4	31357	Waiting to	['Comedy',	['based on	['Whitney	Cheated o	/8MprEuT	127	6.2	
6	5	11862	Father of t	['Comedy',	['daughter	['Steve Ma	Just when	/rj4LBtwQ	106	6.272	
7	6	949	Heat (1995	['Crime', 'D	['robbery',	['Al Pacino	Obsessive	/e09dLw1I	170	7.928	
8	7	11860	Sabrina (1	['Romance	['chauffeu	['Harrison	After her r	/i8PbLJDP	127	6.215	
9	8	45325	Tom and H	['Family', '	['based on	['Jonathan	A mischiev	/bMY31ikE	97	5.3	
10	9	9091	Sudden De	['Action', 'I	['explosive	['Jean-Clau	When a m	/1pylO6YX	110	6	
11	10	710	GoldenEye	['Adventur	['compute	['Pierce Br	When a pc	/z0ljRnNxl	130	6.898	
12	11	9087	American	['Drama', '	['new love	['Michael I	Widowed	/yObOAYF	113	6.534	
13	12	12110	Dracula: D	['Comedy',	['vampire',	['Leslie Nie	When a la	/4rRfZz8Yr	88	6.059	
14	13	21032	Balto (199	['Family', '	['wolf', 'pe	['Kevin Bac	An outcast	/dCVcdb5c	78	7.304	
15	14	10858	Nixon (199	['Drama', '	['usa presi	['Anthony	A look at P	/cz2MTGr	192	6.859	
16	15	1408	Cutthroat	['Action', '	['ship', 'tre	['Geena Da	Morgan Ac	/hYdeBZ4E	124	5.777	
17	16	524	Casino (19	['Crime', 'D	['drug abu	['Robert D	In Las Vega	/gzilkUSnY	179	7.99	

Data set sample

3.5 Data Quality Issues

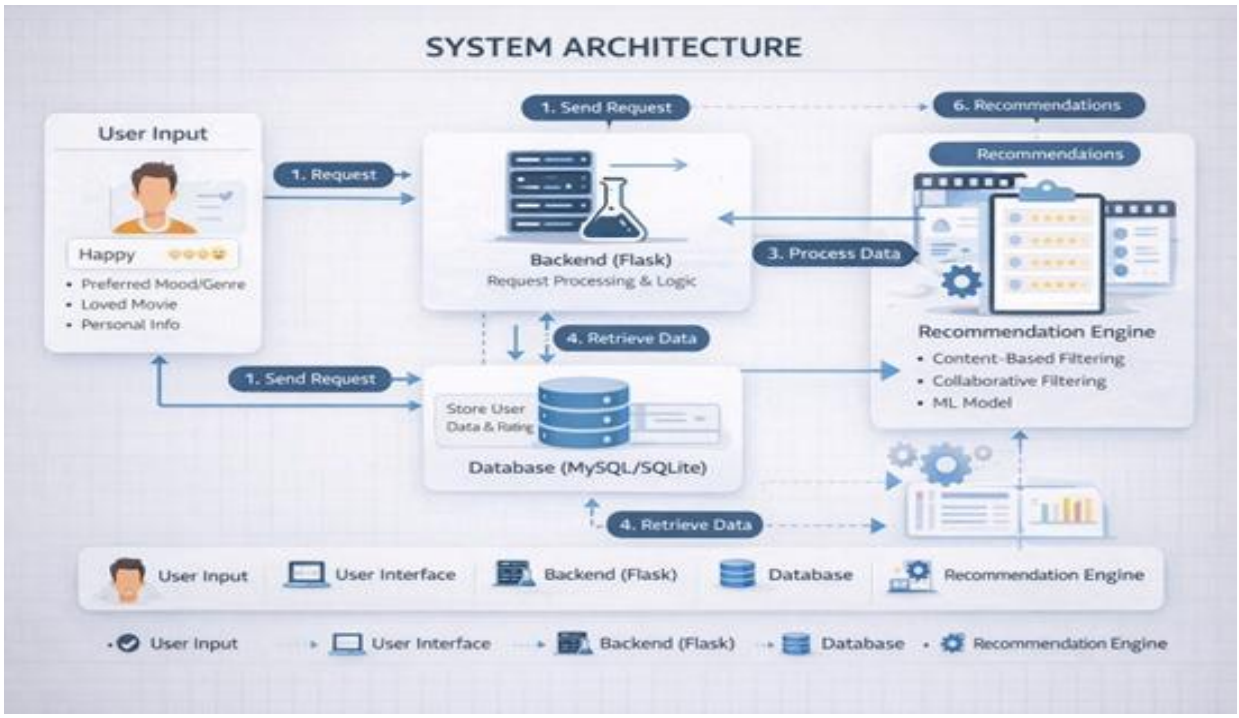
Some data quality issues observed in the dataset include:

• Missing values in movie descriptions or ratings
• Inconsistent genre naming formats
• Duplicate movie entries in some cases
• Imbalance in ratings (popular movies have more ratings)
• Incomplete metadata such as missing cast or keywords
• Presence of noisy or irrelevant data in text fields
• Variations in movie titles causing matching issues

These issues are handled using preprocessing techniques such as data cleaning, normalization, and filtering to improve system performance. Missing values are handled by filling or removing incomplete records, and duplicate entries are eliminated to maintain data consistency. Text data is standardized to ensure uniformity across features, which helps in improving the accuracy and quality of recommendations

CHAPTER 4: METHODOLOGY

4.1 System Architecture



4.1.1 Architectural Overview

The Movie Recommendation System is designed using a **Three-Tier Architecture** to ensure modularity, scalability, and maintainability.

The architecture consists of:

1. Presentation Layer (Frontend)
2. Application Layer (Backend + Machine Learning Engine)
3. Data Layer (Dataset + Database + Model Artifacts)

The core of the system is a **Hybrid Recommendation Engine**, which integrates:

- Content-Based Filtering
- Collaborative Filtering (Matrix Factorization using Truncated SVD)

The hybrid approach enhances recommendation accuracy while mitigating cold-start and sparsity issues.

4.1.2 Architectural Layers

A. Presentation Layer

Technologies Used:

HTML5 – Used to structure and design the content of web pages

CSS3 – Used to style and format the appearance of web pages

Bootstrap 5 – A framework used to create responsive and mobile-friendly designs

Jinja2 Template Engine – Used to dynamically generate HTML pages in Flask

Vanilla JavaScript (AJAX) – Used to send and receive data without reloading the page

Responsibilities:

- User authentication (Login/Register)
- Search autocomplete
- Display trending movies
- Render Netflix-style recommendation cards
- Handle user sessions

The frontend communicates with the backend via RESTful endpoints exposed through Flask.

B. Application Layer

Implemented using the Flask micro-framework.

It handles:

- Business logic
- Recommendation processing
- Model inference
- API integration
- Session management

Core Modules:

Module	Responsibility
app.py	Route handling and session control
content_engine.py	TF-IDF based similarity modeling
collaborative_engine.py	SVD-based matrix factorization
hybrid_engine.py	Weighted score fusion
tmdb_fetcher.py	Metadata enrichment
evaluate_engine.py	Model performance evaluation

C. Data Layer

The Data Layer consists of:

- MovieLens ML-1M Dataset
- SQLite Database (movies_app.db)
- Serialized Machine Learning Models (.pkl files)

Database tables support:

- User authentication
- Rating storage
- Movie metadata caching
- Recommendation history logging

4.1.3 System Workflow

The overall workflow of the proposed movie recommendation system is described as follows:

1.User Authentication:

The user logs into the application using valid credentials to access personalized recommendations.

2.Movie Selection:

The user searches for and selects a preferred movie from the available dataset.

3.Content-Based Processing:

The Content-Based Engine processes the selected movie and retrieves the Top-50 similar movies using TF-IDF vectorization and Cosine Similarity.

4.Collaborative Filtering:

The Collaborative Filtering Engine predicts user-specific ratings by applying the Truncated Singular Value Decomposition (SVD) technique on the user-item interaction matrix.

5. Hybrid Score Computation:

The final recommendation score is calculated by combining both content-based and collaborative scores using a weighted approach:

HybridScore = (0.5 × ContentScore) + (0.3 × CollaborativeScore) + (0.2 × PersonalizationScore)

6.Ranking of Recommendations:

Movies are ranked based on the computed hybrid scores, and the Top-10 highest-ranked movies are selected.

7.Metadata Enrichment:

Additional movie details such as posters, ratings, and descriptions are fetched from the TMDB (The Movie Database) API to enhance the output.

8.User Interface Display:

The final recommendations are presented to the user in a Netflix-style card layout, ensuring an intuitive and visually appealing experience.

4.2 Data Preprocessing

Data preprocessing is an important step in building an effective movie recommendation system. It ensures that the dataset is clean, consistent, and suitable for applying machine learning algorithms. Raw data collected from different sources may contain missing values, duplicates, and inconsistencies, which can affect the performance of the model. Therefore, preprocessing techniques such as data cleaning, handling missing values, feature extraction, and normalization are applied to improve data quality. In this project, movie-related attributes like genres, keywords, cast, and descriptions are processed and transformed into meaningful numerical representations, while user ratings are structured into a user-item matrix. Proper preprocessing helps in improving the accuracy, reliability, and overall performance of the recommendation system.

Data Sources

Source	Purpose
MovieLens ML-1M	User rating matrix construction
TMDB API	Rich metadata enrichment

Dataset Statistics:

- 6,040 Users
- 3,952 Movies
- 1,000,209 Ratings
- Average Rating ≈ 3.58
- Sparsity ≈ 95.8%

4.2.1Data Cleaning

MovieLens dataset uses:

- Custom delimiter (::)

- Latin-1 encoding

  Parsing handled using:

  pd.read_csv(sep='::', engine='python', encoding='latin-1')

  Cleaning Steps:

- Explicit column naming
- Year extraction from movie titles
- Removal of malformed entries
- Encoding normalization
- Metadata token cleanup

4.2.2 Handling Missing Values

Component	Strategy
Rating Matrix	Filled with 0 after mean-centering
Overview field	Replaced with empty string
Poster & Rating	Handled safely in frontend
Text Features	Null values replaced with empty tokens

4.2.3 Encoding

Encoding is the process of converting categorical and textual data into numerical format so that machine learning models can process it effectively. In this project, text data such as movie genres, keywords, cast, and descriptions are encoded using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization. This technique transforms textual features into numerical vectors based on word importance.

Categorical data such as genres and user preferences are also converted into suitable formats for processing. Additionally, user attributes like age and gender are transformed into numerical scores for personalization. Encoding improves the accuracy and efficiency of the recommendation system

4.2.4 Feature Engineering

A. Content Feature Construction

For each movie, multiple attributes such as genres, keywords, cast, and overview are combined to form a single feature representation:

combined_features = genres + keywords + cast + overview

These combined textual features are converted into numerical vectors using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization:

TfidfVectorizer(stop_words='english')

This process generates a high-dimensional sparse matrix, where each movie is represented as a vector of weighted terms. These vectors are later used to compute similarity between movies using cosine similarity.

.B. Collaborative Feature Construction

A user-item interaction matrix is constructed with dimensions:

6040 × 3952

- Rows represent users
- Columns represent movies
- Values represent user ratings

Before applying matrix factorization, the matrix is mean-centered to remove user-specific bias (for example, some users may consistently give higher or lower ratings). This improves the accuracy of predictions generated using the SVD algorithm.

4.2.5 Feature Scaling & Normalization

To ensure compatibility between different components of the recommendation system, normalization is applied to bring all scores to a common scale.

- User ratings are maintained within the range of [1, 5]

- Collaborative filtering predictions are normalized to the range [0, 1]
- Content similarity scores obtained using cosine similarity are naturally in the range [0, 1]
- Personalization scores are also scaled within the range [0, 1]

This normalization ensures that all components contribute proportionally in the hybrid recommendation model and enables effective weighted score combination.

4.2.6 Feature Selection

Feature selection involves identifying the most relevant attributes required for building an effective recommendation system. In this project, important features such as genres, keywords, cast, overview, and user ratings are selected. These features help in representing movie characteristics and user preferences accurately. By focusing on relevant features, the system improves recommendation quality and reduces unnecessary data processing**.**

4.2.7 Descriptive Analytics

Descriptive analytics is used to understand and summarize the dataset before model development. In this project, key statistics such as the number of users, number of movies, total ratings, and average rating are analyzed. This helps in understanding data distribution, user behavior, and sparsity of the dataset, which is essential for designing an efficient recommendation system.

4.3 Model Design

4.3.1 Collaborative Filtering

Collaborative filtering is implemented using the Truncated Singular Value Decomposition (SVD) algorithm, which is widely used in recommendation systems to capture latent relationships between users and items.

The user-item interaction matrix is decomposed as:

$R \approx U\Sigma V^T$

Where:

- U represents the user latent feature matrix
- Σ represents the diagonal matrix of singular values
- V^T represents the item (movie) latent feature matrix

This decomposition helps in reducing dimensionality and identifying hidden patterns in user preferences.

4.3.2 Content-Based Filtering

Content-based filtering recommends movies based on similarity between item features such as genre, keywords, and descriptions.

Algorithms Used:

- TF-IDF Vectorization
- Cosine Similarity

The similarity between movies is computed using cosine similarity:

$$\text{Similarity}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

This metric measures the similarity between two movie vectors. A higher value indicates greater similarity.

4.3.3 Hybrid Recommendation Strategy

To improve recommendation quality, a hybrid approach is used by combining content-based filtering, collaborative filtering, and personalization.

The final recommendation score is calculated as:

$$\text{HybridScore} = (0.5 \times \text{ContentSimilarity}) + (0.3 \times \text{CollaborativePrediction}) + (0.2 \times \text{PersonalizationScore}$$

4.3.4 Justification for Model Selection:

The hybrid recommendation model is selected to combine the strengths of both content-based and collaborative filtering techniques. Content-based filtering ensures that movies are recommended based on similarity in features such as genre, keywords, and description, making it effective for new or less popular movies.

Collaborative filtering, implemented using SVD, captures hidden patterns in user behavior and preferences by analyzing user-item interactions. This helps in providing personalized recommendations based on similar users.

The integration of personalization further enhances the system by incorporating user-specific attributes such as preferences, age, and interests.

The hybrid approach helps in overcoming common limitations such as the cold-start problem and data sparsity, while improving overall recommendation accuracy, diversity, and user satisfaction.

4.3.5 Evaluation Methodology

The performance of the proposed system is evaluated using both quantitative metrics and qualitative validation techniques. Quantitative evaluation is carried out using metrics such as Precision, Recall,RMSE, and MAE to measure the accuracy of predictions and the relevance of recommendations.

RMSE and MAE are used to evaluate the performance of the collaborative filtering model by comparing predicted ratings with actual user ratings. Lower values indicate better prediction accuracy.

Precision, Recall, are used to assess the quality of recommendations by measuring how many recommended movies are relevant to the user and how effectively the system retrieves relevant items.

In addition to quantitative evaluation, qualitative validation is performed through manual inspection of recommended results and self-similarity checks to ensure that the recommendations are meaningful and contextually appropriate.

4.3.6 Collaborative Model Evaluation

The collaborative filtering model is evaluated using **Root Mean Square Error (RMSE)**, which measures the difference between predicted and actual ratings.

RMSE=1n∑(ytrue−ypred)2\text{RMSE}=\sqrt{\frac{1}{n}\sum (y_{true}-y_{pred})^2}RMSE=n1∑(ytrue−ypred)2

A lower RMSE value indicates better prediction accuracy.
In this system, the observed RMSE value is approximately **1.78** indicating satisfactory performance.

4.3.7 Content Model Validation

The content-based model is validated using the following approaches:

Self-Similarity Test:
Ensures that each movie is most similar to itself, validating feature extraction accuracy

Manual Inspection:

The recommended movies are manually reviewed to verify relevance and contextual similarity

These methods confirm that the system generates meaningful and accurate recommendations.

4.4 Tools and Technologies

The development of the Movie Recommendation System involves the use of various tools and technologies for data processing, model building, and application development.

Programming Language

- **Python:** Used as the primary programming language for implementing machine learning algorithms and backend logic.

Libraries

- **NumPy:** Used for numerical computations and matrix operations
- **Pandas:** Used for data manipulation and preprocessing
- **Scikit-learn:** Used for implementing TF-IDF vectorization, cosine similarity, and SVD-based collaborative filtering

Framework

- **Flask:** A lightweight web framework used to build the backend and handle requests, responses, and API integration

Frontend Technologies

- **HTML5, CSS3, Bootstrap:** Used to design the user interface
- **JavaScript (AJAX):** Used for dynamic content updates and interaction

Database

- **SQLite / MySQL:** Used to store user data, movie information, and ratings

Dataset and APIs

- **MovieLens Dataset:** Used for user ratings and movie data
- **TMDB API:** Used to fetch additional movie details such as posters and descriptions

Development Tools

- **Jupyter Notebook / VS Code:** Used for development, testing, and experimentation
- **Web Browser:** Used to run and test the application.

CHAPTER 5: IMPLEMENTATION

Overview

The implementation of the Movie Recommendation System is carried out using Python and Flask framework. The system integrates content-based filtering, collaborative filtering, and a hybrid recommendation approach to generate personalized movie suggestions. The implementation includes data processing, model building, backend development, and user interface design.

5.1 Module-wise Implementation

The system is divided into several modules, each responsible for specific functionalities:

1. User Module

- Handles user registration and login
- Stores user information and preferences
- Manages user sessions

2. Data Processing Module

- Loads movie and ratings datasets
- Performs preprocessing such as cleaning, encoding, and feature extraction
- Prepares data for recommendation models

3. Content-Based Recommendation Module

- Combines movie features such as genres, keywords, cast, and overview
- Applies TF-IDF vectorization
- Computes cosine similarity between movies
- Generates similar movie recommendations

4. Collaborative Filtering Module

- Constructs user-item interaction matrix
- Applies Truncated SVD for matrix factorization
- Predicts user ratings for unseen movies

5. Hybrid Recommendation Module

- Combines content-based, collaborative, and personalization scores
- Uses weighted formula to generate final recommendations
- Ranks movies based on hybrid scores

6. API and Backend Module

- Implemented using Flask
- Handles HTTP requests and responses
- Connects frontend with recommendation engine

7. Database Module

- Stores user details, ratings, and movie data
- Uses SQLite/MySQL for data storage

5.2 Algorithm Steps (Pseudocode)

Step1: Load movie dataset and user ratings

Step2: Preprocess data (cleaning, encoding, feature engineering)

Step3: Generate TF-IDF matrix for movie features

Step4: Compute cosine similarity between moviesO

Step5: Create user-item matrix for collaborative filtering

Step6: Apply SVD to predict missing ratings

Step7: Normalize scores (content, collaborative, personalization)

Step8: Compute hybrid score

Step9: Rank movies based on hybrid score

Step 10: Display Top-N recommendations to user

5.3 Code snippets:-

APP.PY

1. App Initialization

```
from flask import Flask, render_template, request, jsonify, session, redirect, url_for
import os
import json
from models.hybrid_engine import HybridEngine
from db_utils import get_db_connection
from werkzeug.security import generate_password_hash, check_password_hash
from dotenv import load_dotenv

load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv("FLASK_SECRET_KEY", "supersecretmoviekey")

@app.template_filter('from_json')
def from_json_filter(value):
    try:
        return json.loads(value or '[]')
    except (json.JSONDecodeError, TypeError):
        return []

hybrid = HybridEngine()

@app.route('/')
def index():
```

```
    if 'user_id' not in session:
        return redirect(url_for('login'))

    conn = get_db_connection()
    trending = conn.execute(
        'SELECT * FROM movies ORDER BY vote_average DESC LIMIT 20'
    ).fetchall()

    featured = conn.execute(
        'SELECT * FROM movies WHERE vote_average > 8 ORDER BY RANDOM() LIMIT 1'
    ).fetchone()

    user = conn.execute(
        'SELECT * FROM users WHERE user_id = ?', (session['user_id'],)
    ).fetchone()

    conn.close()

    profile_incomplete = not (user['age'] and user['gender'] and user['preferred_genres'])

    return render_template(
        'index.html',
        trending=trending,
        featured=featured,
        user=user,
        profile_incomplete=profile_incomplete
```

```
    )

@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None

    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        conn = get_db_connection()
        user = conn.execute(
            'SELECT * FROM users WHERE username = ?', (username,)
        ).fetchone()
        conn.close()

        if user and check_password_hash(user['password_hash'], password):
            session['user_id'] = user['user_id']
            session['username'] = user['username']
            session['full_name'] = user['full_name'] or user['username']
            return redirect(url_for('index'))
        else:
            error = "Invalid username or password"

    return render_template('login.html', error=error)

@app.route('/register', methods=['GET', 'POST'])
```

```
def register():
    error = None

    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')

        full_name = request.form.get('full_name', '').strip() or None
        age_raw = request.form.get('age', '').strip()
        age = int(age_raw) if age_raw.isdigit() else None
        gender = request.form.get('gender') or None

        preferred_genres_list = request.form.getlist('preferred_genres')
        preferred_genres_json = json.dumps(preferred_genres_list) if preferred_genres_list
else None

        conn = get_db_connection()

        try:
            hashed_pw = generate_password_hash(password)
            conn.execute(
                '''INSERT INTO users
                    (username, email, password_hash, full_name, age, gender, preferred_genres)
                    VALUES (?, ?, ?, ?, ?, ?, ?)''',
                (username, email, hashed_pw, full_name, age, gender, preferred_genres_json)
            )
```

```
            conn.commit()
            conn.close()
            return redirect(url_for('login'))

        except Exception:
            error = "Username or Email already exists. Please try again."
            conn.close()

    return render_template('register.html', error=error)

@app.route('/profile', methods=['GET', 'POST'])
def profile():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    conn = get_db_connection()

    user = conn.execute(
        'SELECT * FROM users WHERE user_id = ?', (session['user_id'],)
    ).fetchone()

    success = False
    error = None

    if request.method == 'POST':
        full_name = request.form.get('full_name', '').strip() or None
        age_raw = request.form.get('age', '').strip()
```

```
        age = int(age_raw) if age_raw.isdigit() else None
        gender = request.form.get('gender') or None

        preferred_genres_list = request.form.getlist('preferred_genres')
        preferred_genres_json = json.dumps(preferred_genres_list) if preferred_genres_list else None

        try:
            conn.execute(
                '''UPDATE users
                    SET full_name = ?, age = ?, gender = ?, preferred_genres = ?
                    WHERE user_id = ?''',
                (full_name, age, gender, preferred_genres_json, session['user_id'])
            )
            conn.commit()

            user = conn.execute(
                'SELECT * FROM users WHERE user_id = ?', (session['user_id'],)
            ).fetchone()

            session['full_name'] = full_name or user['username']
            success = True

        except Exception:
            error = "Could not save profile. Please try again."

    conn.close()
```

```
    return render_template('profile.html', user=user, success=success, error=error)

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

@app.route('/search_movies')
def search_movies():
    query = request.args.get('q', '')

    conn = get_db_connection()
    movies = conn.execute(
        'SELECT title FROM movies WHERE title LIKE ? LIMIT 5',
        (f'%{query}%',)
    ).fetchall()
    conn.close()

    return jsonify([m['title'] for m in movies])

@app.route('/recommend', methods=['POST'])
def recommend():
    if 'user_id' not in session:
        return jsonify({"error": "Unauthorized"}), 401

    data = request.json
    favorite_movie = data.get('favorite_movie')
```

```
    if not favorite_movie:
        return jsonify({"error": "No movie selected"}), 400

    conn = get_db_connection()

    user = conn.execute(
        'SELECT * FROM users WHERE user_id = ?', (session['user_id'],)
    ).fetchone()

    user_profile = dict(user) if user else {}

    try:
        recommendations = hybrid.get_hybrid_recommendations(
            session['user_id'],
            favorite_movie,
            user_profile=user_profile
        )

        conn.close()
        return jsonify(recommendations)

    except Exception as e:
        conn.close()
        return jsonify({"error": str(e)}), 500
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

CONTENT BASED FILTERING

```
import os

import pandas as pd

import pickle

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity


class ContentEngine:

    def __init__(self, metadata_path='data/movie_metadata_checkpoint.csv',
model_path='models/content_model.pkl'):

        self.metadata_path = metadata_path

        self.model_path = model_path

        self.tfidf_matrix = None

        self.vectorizer = TfidfVectorizer(stop_words='english')

        self.movies_df = None


    def prepare_data(self):

        if not os.path.exists(self.metadata_path):

            print(f"Error: Metadata file {self.metadata_path} not found.")

            return False


        self.movies_df = pd.read_csv(self.metadata_path)


        # Combine features (genres, keywords, cast, overview)

        def clean_data(x):

            if isinstance(x, str):

                return x.replace('[', '').replace(']', '').replace('"', '').replace(',', ' ')
```

```
        return ''

    self.movies_df['combined_features'] = (
        self.movies_df['genres'].apply(clean_data) + ' ' +
        self.movies_df['keywords'].apply(clean_data) + ' ' +
        self.movies_df['cast'].apply(clean_data) + ' ' +
        self.movies_df['overview'].fillna('')
    )

    print("Vectorizing movie features...")
    self.tfidf_matrix = self.vectorizer.fit_transform(self.movies_df['combined_features'])
    return True

def save_model(self):
    if not os.path.exists('models'):
        os.makedirs('models')

    data_to_save = {
        'tfidf_matrix': self.tfidf_matrix,
        'vectorizer': self.vectorizer,
        'movies_df': self.movies_df
    }

    with open(self.model_path, 'wb') as f:
        pickle.dump(data_to_save, f)
    print(f"Content model saved to {self.model_path}")
```

```
def load_model(self):
    if os.path.exists(self.model_path):
        with open(self.model_path, 'rb') as f:
            data = pickle.load(f)
            self.tfidf_matrix = data['tfidf_matrix']
            self.vectorizer = data['vectorizer']
            self.movies_df = data['movies_df']

        print("Content model loaded successfully.")
        return True

    return False

def get_similar_movies(self, movie_title, top_n=10):
    if self.movies_df is None:
        if not self.load_model():
            return []

    import re
    safe_title = re.escape(movie_title.strip())

    # Find movie index
    idx_list = self.movies_df.index[
        self.movies_df['title'].str.contains(safe_title, case=False, na=False, regex=True)
    ].tolist()
    if not idx_list:
        idx_list = [
```

```
            i for i, t in enumerate(self.movies_df['title'].tolist())
            if movie_title.lower() in t.lower()
        ]

    if not idx_list:
        return []

    idx = idx_list[0]

    # Compute similarity
    sim_scores = list(enumerate(cosine_similarity(self.tfidf_matrix[idx],
self.tfidf_matrix)[0]))

    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:top_n+1]

    movie_indices = [i[0] for i in sim_scores]
    scores = [i[1] for i in sim_scores]

    results = []
    for i, score in zip(movie_indices, scores):
        results.append({
            'movie_id': int(self.movies_df.iloc[i]['movie_id']),
            'title': self.movies_df.iloc[i]['title'],
            'score': float(score),
            'genres': str(self.movies_df.iloc[i].get('genres', '') or '')
        })
```

```
        return results
if __name__ == "__main__":
    import sys
    sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

    engine = ContentEngine()

    if engine.prepare_data():
        engine.save_model()
        print(engine.get_similar_movies("Toy Story", top_n=5))
```

COLLABRATIVE BASED MODEL:

```
import os
import pickle
import pandas as pd
import numpy as np
from sklearn.decomposition import TruncatedSVD

class CollaborativeEngine:
    def __init__(self, model_path='models/svd_model_sklearn.pkl'):
        self.model_path = model_path
        self.model = None
        self.user_item_matrix = None
        self.user_index = None
        self.item_index = None
        self.reconstructed_df = None
        self.user_means = None
```

```
def train(self, ratings_df):
    print("Training Collaborative Filtering model...")

    self.user_item_matrix = ratings_df.pivot(
        index='user_id', columns='movie_id', values='rating'
    ).fillna(0)

    self.user_index = self.user_item_matrix.index
    self.item_index = self.user_item_matrix.columns

    self.user_means = self.user_item_matrix.replace(0, np.nan).mean(axis=1)
    matrix_centered = self.user_item_matrix.sub(self.user_means, axis=0).fillna(0)

    self.model = TruncatedSVD(n_components=150, n_iter=10, random_state=42)
    matrix_reduced = self.model.fit_transform(matrix_centered)

    reconstructed_centered = np.dot(matrix_reduced, self.model.components_)
    reconstructed = reconstructed_centered + self.user_means.values.reshape(-1, 1)

    reconstructed = np.clip(reconstructed, 1, 5)

    self.reconstructed_df = pd.DataFrame(
        reconstructed, index=self.user_index, columns=self.item_index
    )

    print("Model trained successfully.")
    self.save_model()
```

```
def save_model(self):
    if not os.path.exists('models'):
        os.makedirs('models')

    data_to_save = {
        'model': self.model,
        'reconstructed_df': self.reconstructed_df,
        'user_index': self.user_index,
        'item_index': self.item_index,
        'user_means': self.user_means
    }

    with open(self.model_path, 'wb') as f:
        pickle.dump(data_to_save, f)

    print(f"Model saved to {self.model_path}")

def load_model(self):
    if os.path.exists(self.model_path):
        with open(self.model_path, 'rb') as f:
            data = pickle.load(f)
            self.model = data['model']
            self.reconstructed_df = data['reconstructed_df']
            self.user_index = data['user_index']
            self.item_index = data['item_index']
            self.user_means = data.get('user_means', None)
```

```
            print("Model loaded successfully.")
            return True

        return False

    def predict(self, user_id, movie_id):
        if self.reconstructed_df is not None:
            if user_id in self.reconstructed_df.index and movie_id in self.reconstructed_df.columns:
                return float(self.reconstructed_df.loc[user_id, movie_id])

        return 3.0

    def get_recommendations(self, user_id, top_n=10):
        if self.reconstructed_df is None:
            if not self.load_model():
                return []

        if user_id not in self.reconstructed_df.index:
            return []

        user_predictions = self.reconstructed_df.loc[user_id].sort_values(ascending=False)
        return user_predictions.head(top_n).index.tolist()

if __name__ == "__main__":
    import sys
    sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
```

```
    from data_loader import load_data

    _, ratings, _ = load_data()

    engine = CollaborativeEngine()
    engine.train(ratings)

    print("Training complete!")
```

Hybrid Engine

```
import os
import pandas as pd
import numpy as np

if __name__ == "__main__":
    import sys
    sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
    from models.content_engine import ContentEngine
    from models.collaborative_engine import CollaborativeEngine
    from models.personalization_engine import PersonalizationEngine
else:
    from .content_engine import ContentEngine
    from .collaborative_engine import CollaborativeEngine
    from .personalization_engine import PersonalizationEngine

class HybridEngine:
```

```
    def __init__(self, content_weight=0.50, collab_weight=0.30, persona_weight=0.20):
        self.content_engine = ContentEngine()
        self.collab_engine = CollaborativeEngine()
        self.persona_engine = PersonalizationEngine()
        self.content_weight = content_weight
        self.collab_weight = collab_weight
        self.persona_weight = persona_weight

    def load_engines(self):
        content_loaded = self.content_engine.load_model()
        collab_loaded = self.collab_engine.load_model()
        return content_loaded and collab_loaded

    def get_hybrid_recommendations(self, user_id, movie_title, top_n=10,
user_profile=None):
        content_recs = self.content_engine.get_similar_movies(movie_title, top_n=50)
        if not content_recs:
            return []

        hybrid_scores = []
        for rec in content_recs:
            m_id = rec['movie_id']
            c_score = rec['score']

            collab_pred = self.collab_engine.predict(user_id, m_id)
            cf_score = collab_pred / 5.0
```

```
            movie_genres_str = rec.get('genres', '')
            if user_profile:
                p_score = self.persona_engine.persona_score(movie_genres_str, user_profile)
            else:
                p_score = 0.5

            final_score = (
                self.content_weight * c_score +
                self.collab_weight * cf_score +
                self.persona_weight * p_score
            )

            hybrid_scores.append({
                'movie_id': m_id,
                'title': rec['title'],
                'hybrid_score': round(final_score, 4),
                'content_score': round(c_score, 4),
                'collab_score': round(cf_score, 4),
                'persona_score': round(p_score, 4),
                'genres': movie_genres_str,
            })

        hybrid_scores.sort(key=lambda x: x['hybrid_score'], reverse=True)
        return hybrid_scores[:top_n]


if __name__ == "__main__":
```

```
    hybrid = HybridEngine()
    if hybrid.load_engines():
        print("Engines loaded. Testing...")
        profile = {"age": 14, "gender": "male", "preferred_genres": '["Animation","Adventure"]'}
        recs = hybrid.get_hybrid_recommendations(1, "Toy Story", top_n=5, user_profile=profile)
        for r in recs:
            print(f"{r['title']:40s}  hybrid={r['hybrid_score']:.4f}  "
                  f"content={r['content_score']:.4f}  collab={r['collab_score']:.4f}  "
                  f"persona={r['persona_score']:.4f}")
    else:
        print("Engines not ready.")
```

PERSONALIZATION ENGINE

```
import json

ALL_GENRES = [
    "Action", "Adventure", "Animation", "Comedy", "Crime",
    "Documentary", "Drama", "Fantasy", "Horror", "Mystery",
    "Romance", "Science Fiction", "Sci-Fi", "Thriller", "War", "Western", "Family"
]

AGE_GENRE_AFFINITY = {
    "under_13": {
        "Animation": 1.0, "Family": 1.0, "Adventure": 0.7, "Comedy": 0.6,
        "Fantasy": 0.5, "Horror": -1.0, "Thriller": -0.8, "Crime": -0.6,
```

```
        "War": -0.5
    },
    "teen": {
        "Animation": 0.6, "Adventure": 0.8, "Action": 0.7, "Fantasy": 0.7,
        "Comedy": 0.5, "Horror": 0.3, "Sci-Fi": 0.5, "Science Fiction": 0.5,
        "Drama": 0.2
    },
    "young_adult": {
        "Action": 0.5, "Horror": 0.6, "Thriller": 0.6, "Sci-Fi": 0.5,
        "Science Fiction": 0.5, "Comedy": 0.4, "Romance": 0.4, "Fantasy": 0.4,
        "Adventure": 0.4
    },
    "adult": {
        "Drama": 0.6, "Thriller": 0.5, "Mystery": 0.5, "Crime": 0.4,
        "War": 0.3, "Documentary": 0.4, "Romance": 0.3
    },
    "senior": {
        "Drama": 0.7, "Documentary": 0.7, "History": 0.6, "War": 0.5,
        "Mystery": 0.4, "Horror": -0.5, "Action": -0.2
    }
}

GENDER_GENRE_AFFINITY = {
    "male": {
        "Action": 0.6, "Adventure": 0.5, "Sci-Fi": 0.5, "Science Fiction": 0.5,
        "War": 0.5, "Crime": 0.4, "Thriller": 0.3, "Western": 0.4,
        "Romance": -0.2
```

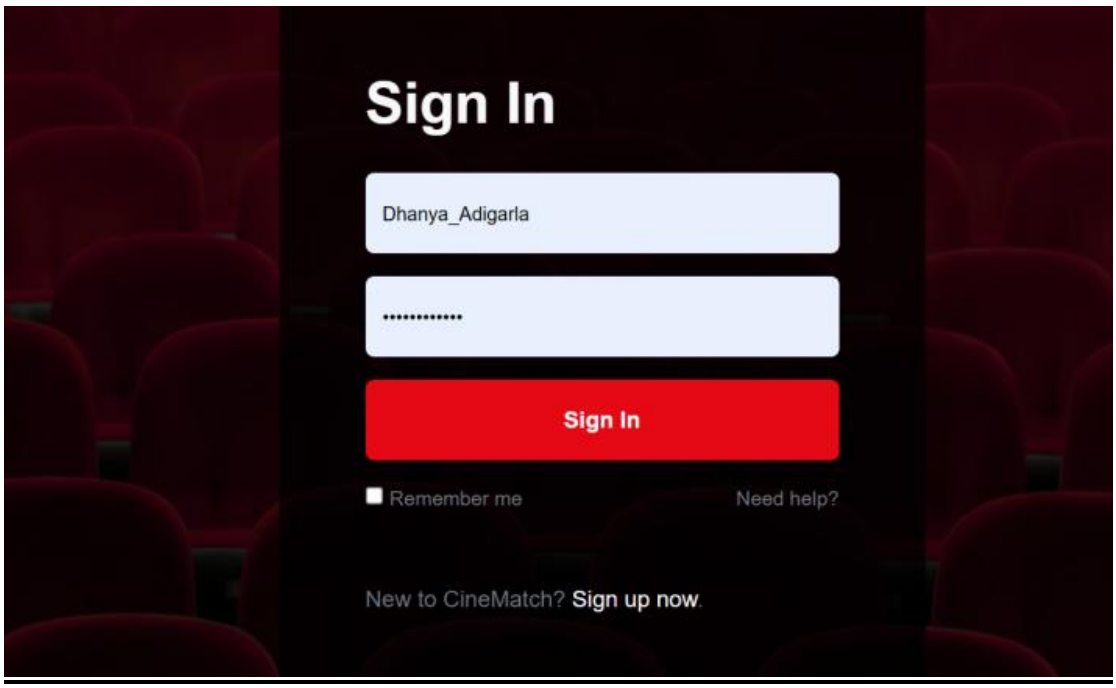
```
    },
    "female": {
        "Romance": 0.6, "Drama": 0.5, "Animation": 0.4, "Comedy": 0.4,
        "Fantasy": 0.4, "Family": 0.3, "Mystery": 0.3, "Thriller": 0.2,
        "War": -0.2
    },
    "other": {},
    "prefer_not_to_say": {}
}


def _parse_genres(genre_str: str) -> list:
    if not genre_str:
        return []
    s = genre_str.strip()
    if s.startswith('['):
        import json, ast
        try:
            return [g.strip() for g in json.loads(s)]
        except:
            try:
                return [g.strip() for g in ast.literal_eval(s)]
            except:
                pass
    s = s.replace('|', ',')
    return [g.strip().strip('"\'') for g in s.split(',') if g.strip().strip('"\'')]


def _age_bracket(age: int) -> str:
```

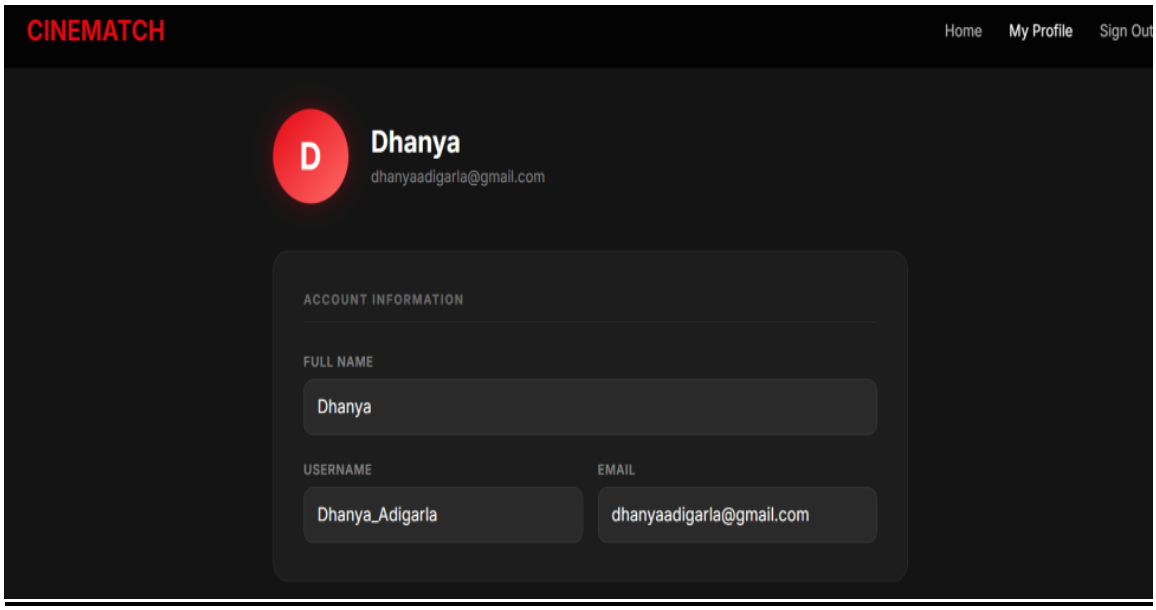
```
    if age < 13:
        return "under_13"
    elif age < 18:
        return "teen"
    elif age < 31:
        return "young_adult"
    elif age < 51:
        return "adult"
    else:
        return "senior"

class PersonalizationEngine:

    def _genre_preference_score(self, movie_genres: list, preferred: list) -> float:
        if not preferred:
            return 0.5

        movie_set = set(g.lower() for g in movie_genres)
        pref_set = set(g.lower() for g in preferred)
        matches = len(movie_set & pref_set)
        return min(matches / len(pref_set), 1.0)

    def _age_score(self, age: int, movie_genres: list) -> float:
        bracket = _age_bracket(age)
        affinity = AGE_GENRE_AFFINITY.get(bracket, {})
        if not affinity or not movie_genres:
            return 0.5
```

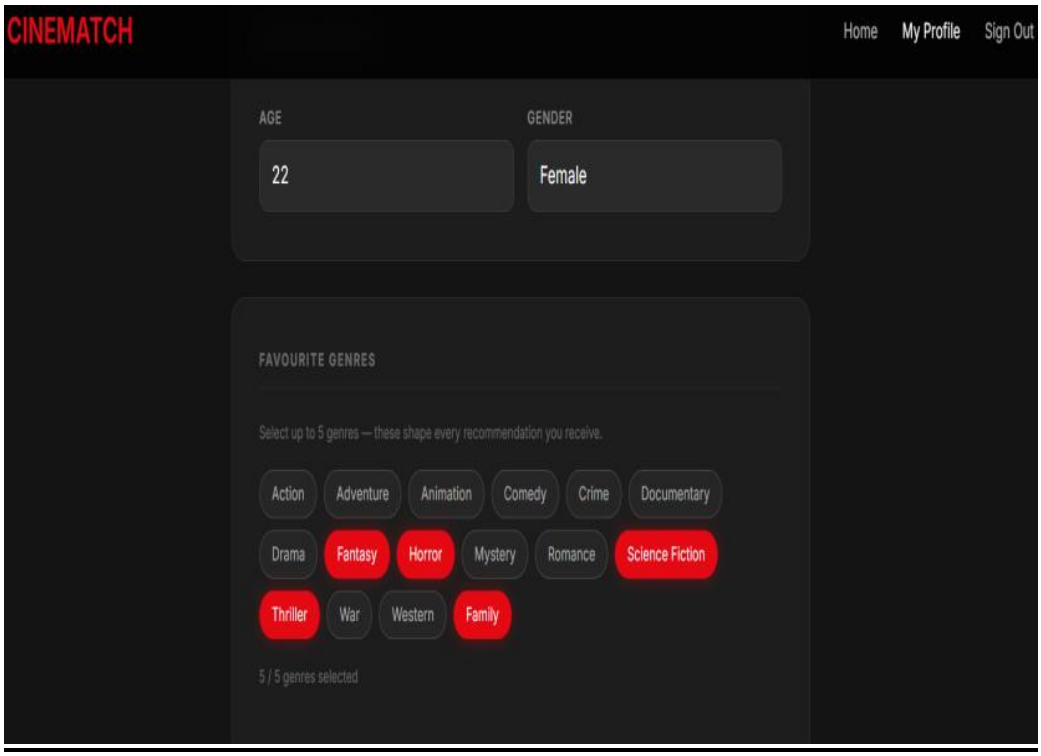
```
    scores = []
    for genre in movie_genres:
        scores.append(affinity.get(genre, 0.3))

    avg = sum(scores) / len(scores)
    return max(0.0, min(1.0, (avg + 1.0) / 2.0))

def _gender_affinity_score(self, gender: str, movie_genres: list) -> float:
    affinity = GENDER_GENRE_AFFINITY.get((gender or "").lower(), {})
    if not affinity or not movie_genres:
        return 0.5

    scores = []
    for genre in movie_genres:
        scores.append(affinity.get(genre, 0.0))

    avg = sum(scores) / len(scores)
    return max(0.0, min(1.0, (avg + 1.0) / 2.0))

def persona_score(self, movie_genres_str: str, user_profile: dict) -> float:
    movie_genres = _parse_genres(movie_genres_str)

    raw_pref = user_profile.get("preferred_genres") or "[]"
    try:
        preferred = json.loads(raw_pref)
    except:
        preferred = []
```

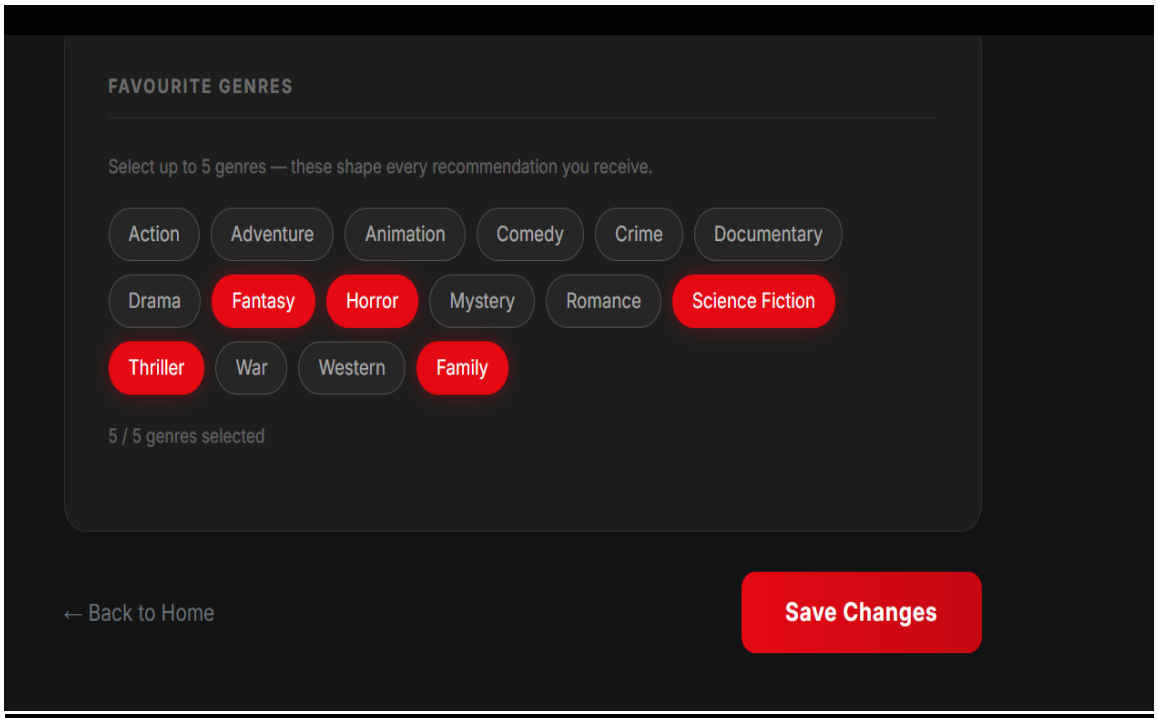
```
age = int(user_profile.get("age") or 25)
gender = str(user_profile.get("gender") or "prefer_not_to_say")

genre_score = self._genre_preference_score(movie_genres, preferred)
age_score = self._age_score(age, movie_genres)
gender_score = self._gender_affinity_score(gender, movie_genres)

final = (0.50 * genre_score) + (0.30 * age_score) + (0.20 * gender_score)
return round(final, 4)
```

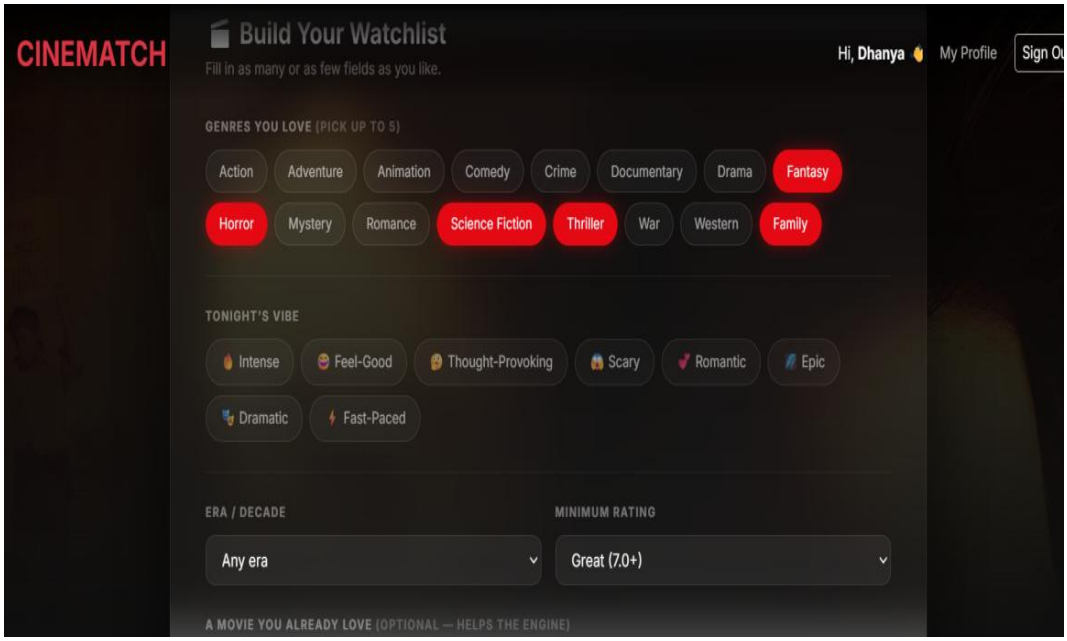
5.4 SCREENSHOTS OF OUTPUTS:

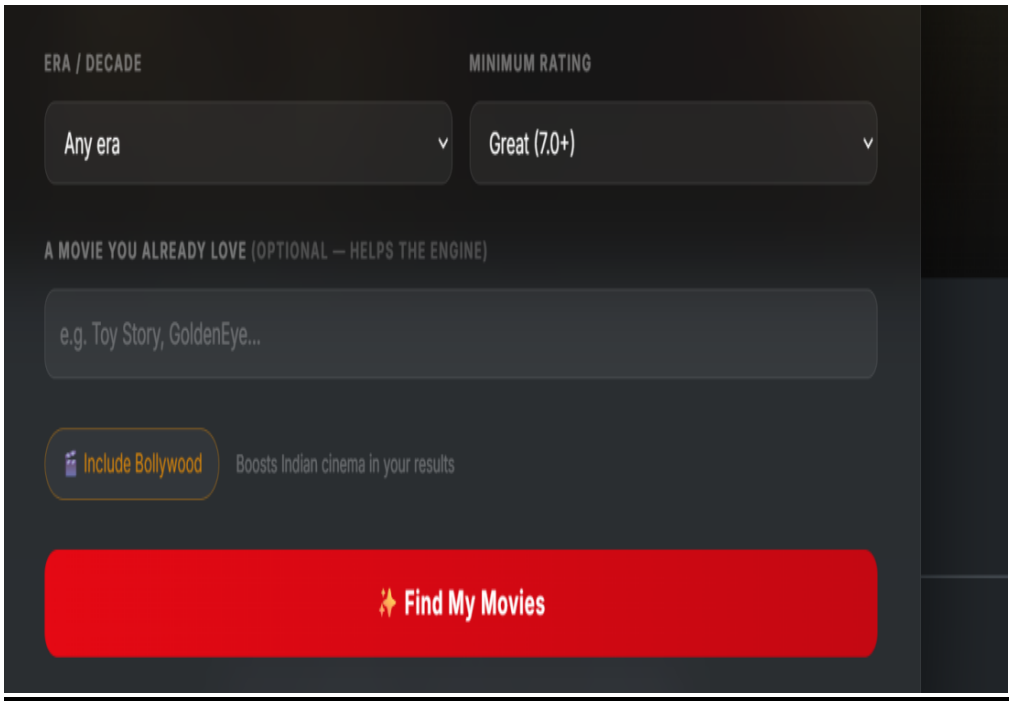


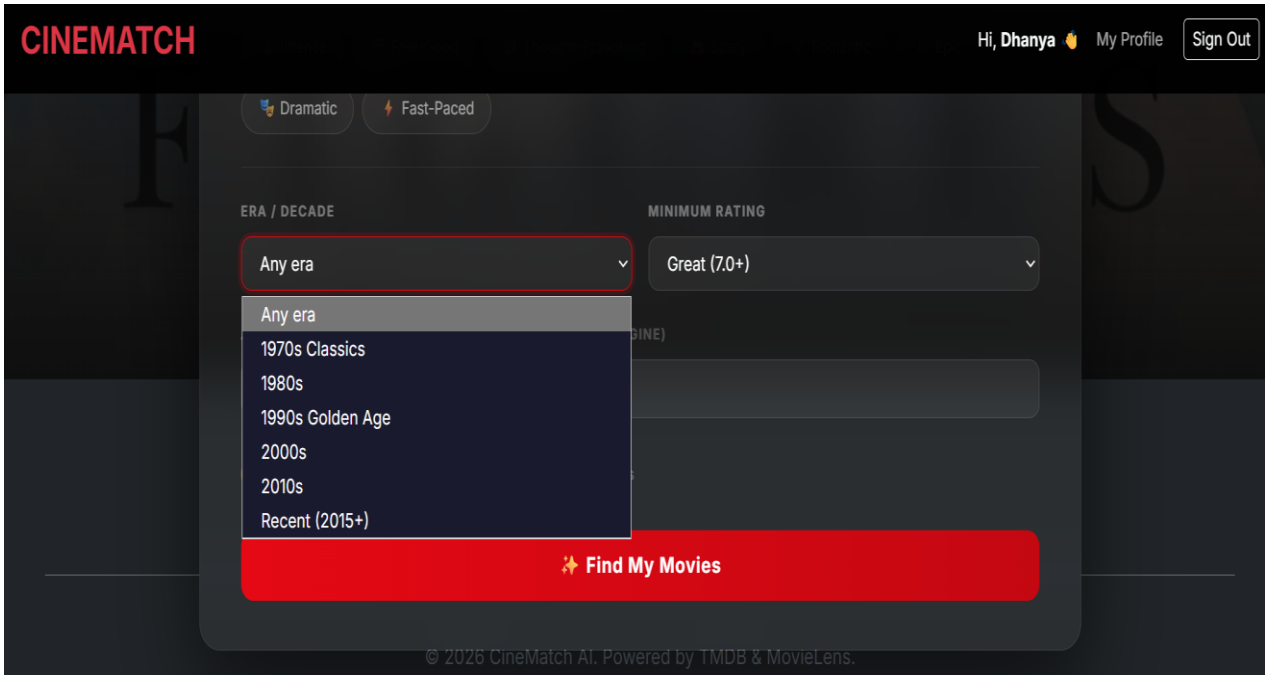

CINEMATCH
Home
My Profile
Sign Out
D
Dhanya
dhanyaadigarla@gmail.com
ACCOUNT INFORMATION
FULL NAME
Dhanya
USERNAME
Dhanya_Adigarla
EMAIL
dhanyaadigarla@gmail.com

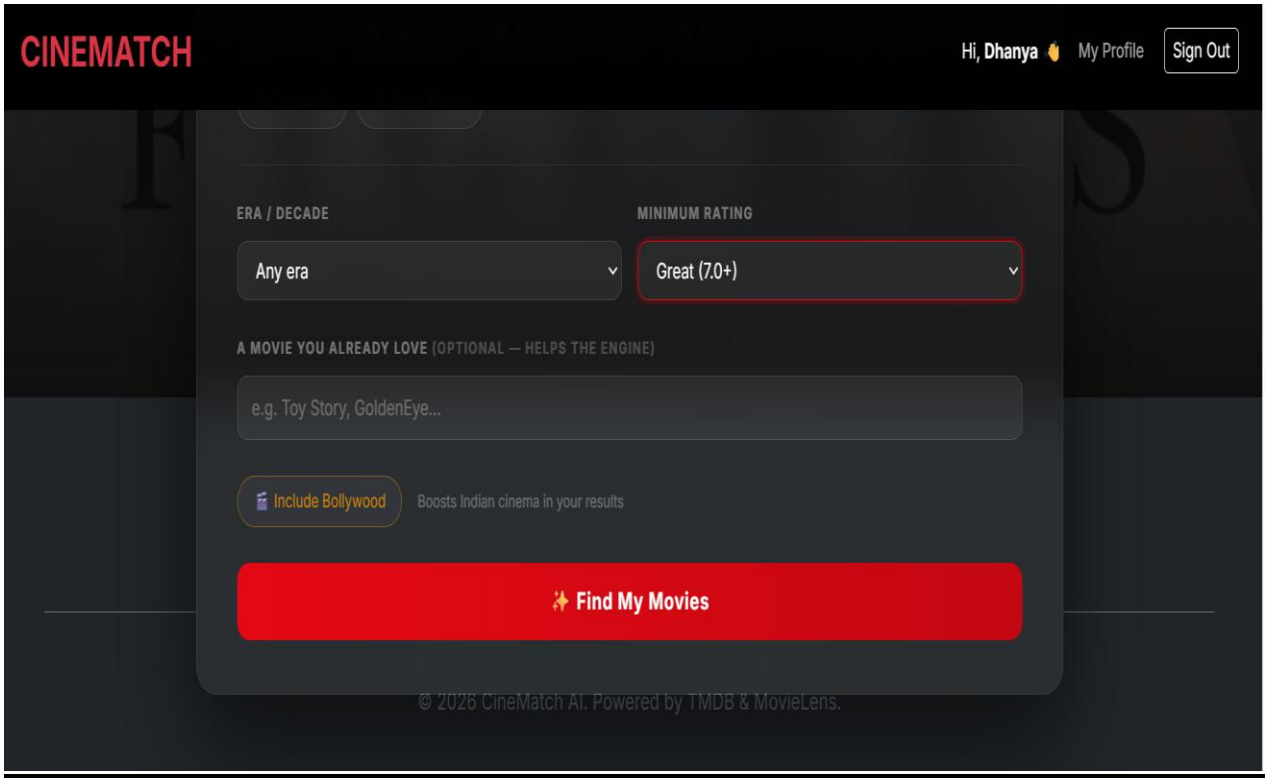

CINEMATCH
Home
My Profile
Sign Out
AGE
22
GENDER
Female
FAVOURITE GENRES
Select up to 5 genres — these shape every recommendation you receive.
Action
Adventure
Animation
Comedy
Crime
Documentary
Drama
Fantasy
Horror
Mystery
Romance
Science Fiction
Thriller
War
Western
Family
5 / 5 genres selected

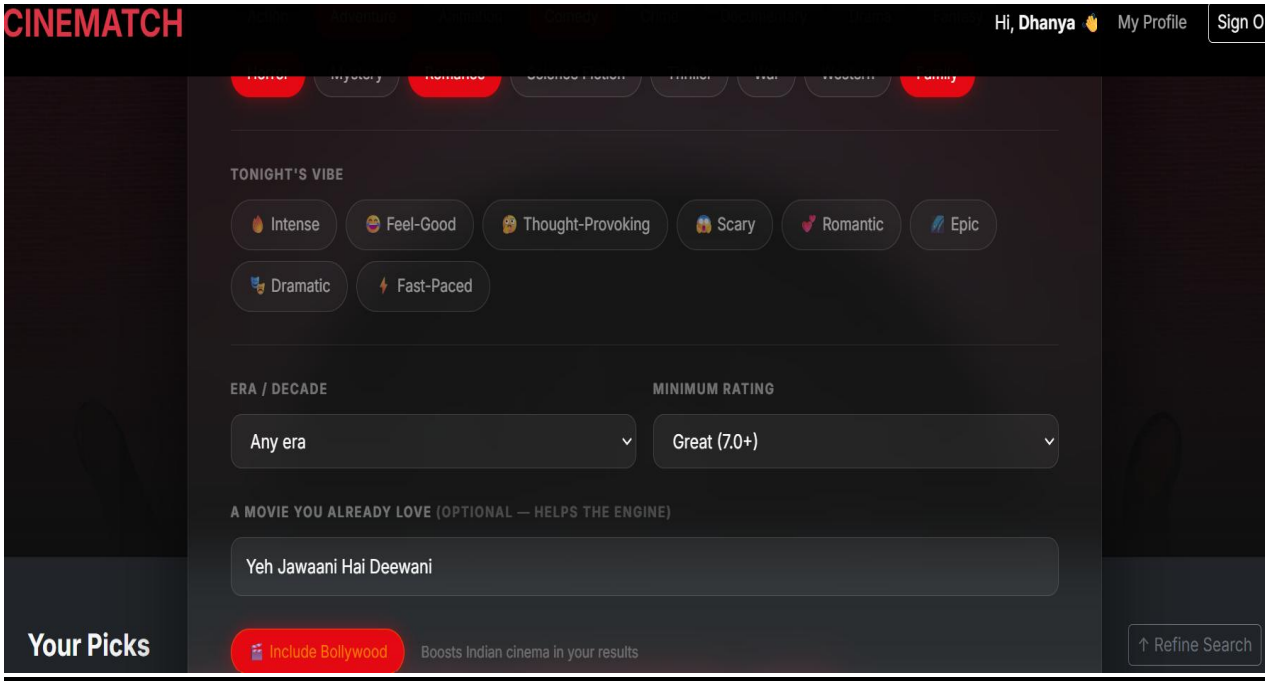

FAVOURITE GENRES
Select up to 5 genres — these shape every recommendation you receive.
Action
Adventure
Animation
Comedy
Crime
Documentary
Drama
Fantasy
Horror
Mystery
Romance
Science Fiction
Thriller
War
Western
Family
5 / 5 genres selected
← Back to Home
Save Changes


CINEMATCH
Build Your Watchlist
Fill in as many or as few fields as you like.
Hi, Dhanya
My Profile
GENRES YOU LOVE (PICK UP TO 5)
Action
Adventure
Animation
Comedy
Crime
Documentary
Drama
Fantasy
Horror
Mystery
Romance
Science Fiction
Thriller
War
Western
Family
TONIGHT'S VIBE
Intense
Feel-Good
Thought-Provoking
Scary
Romantic
Epic
Dramatic
Fast-Paced
ERA / DECADE
Any era
MINIMUM RATING
Great (7.0+)
A MOVIE YOU ALREADY LOVE (OPTIONAL — HELPS THE ENGINE)

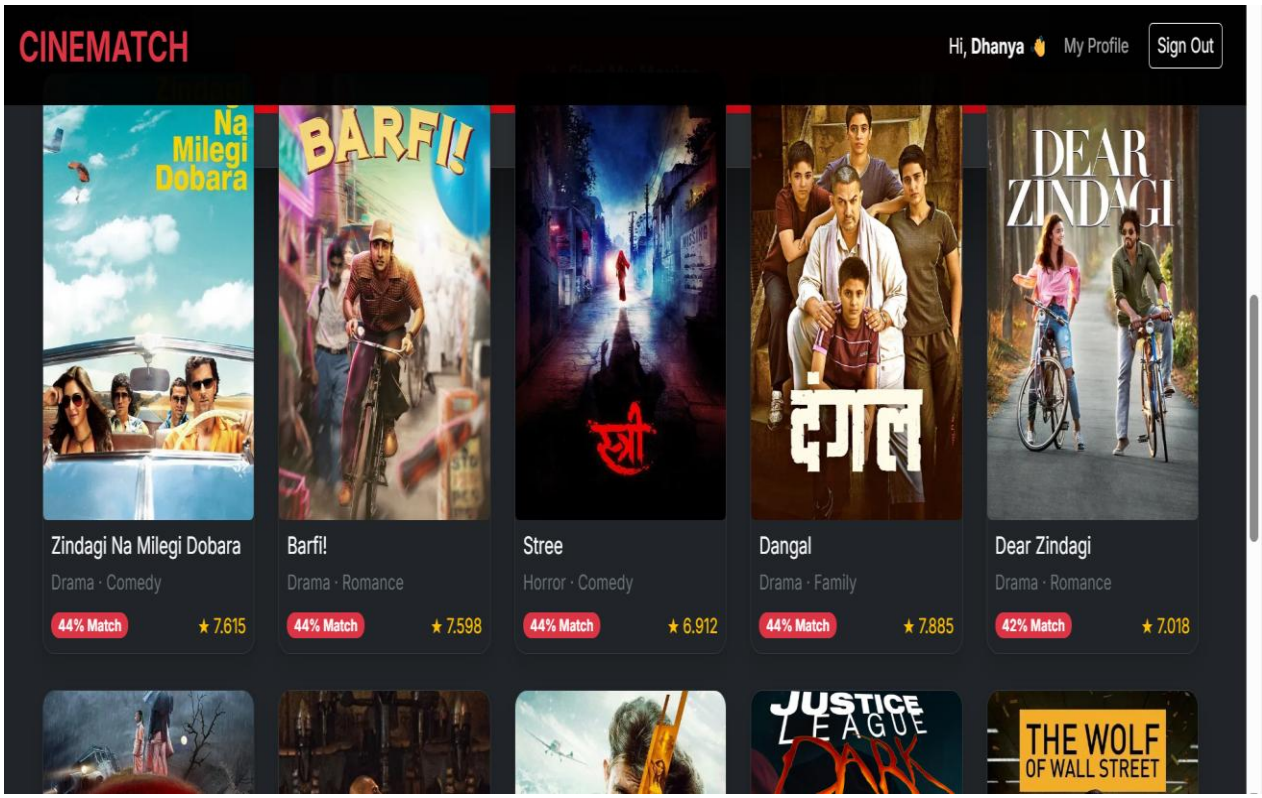

ERA / DECADE
MINIMUM RATING
Any era
Great (7.0+)
A MOVIE YOU ALREADY LOVE (OPTIONAL — HELPS THE ENGINE)
e.g. Toy Story, GoldenEye...
Include Bollywood
Boosts Indian cinema in your results
Find My Movies

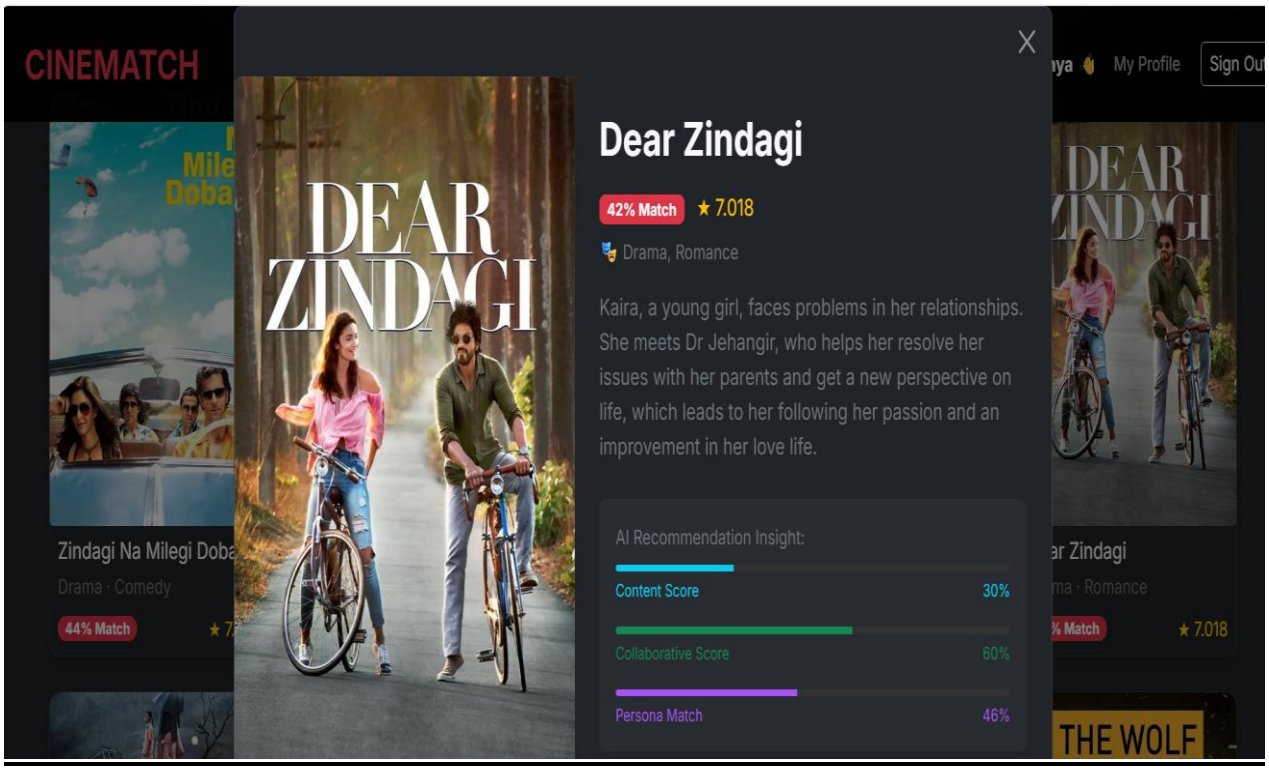

CINEMATCH
Hi, Dhanya
My Profile
Sign Out
Dramatic
Fast-Paced
ERA / DECADE
MINIMUM RATING
Any era
Great (7.0+)
Any era
1970s Classics
1980s
1990s Golden Age
2000s
2010s
Recent (2015+)
Find My Movies
© 2026 CineMatch AI. Powered by TMDB & MovieLens.


CINEMATCH
Hi, Dhanya
My Profile
Sign Out
ERA / DECADE
Any era
MINIMUM RATING
Great (7.0+)
A MOVIE YOU ALREADY LOVE (OPTIONAL — HELPS THE ENGINE)
e.g. Toy Story, GoldenEye...
Include Bollywood
Boosts Indian cinema in your results
Find My Movies
© 2026 CineMatch AI. Powered by TMDB & MovieLens.

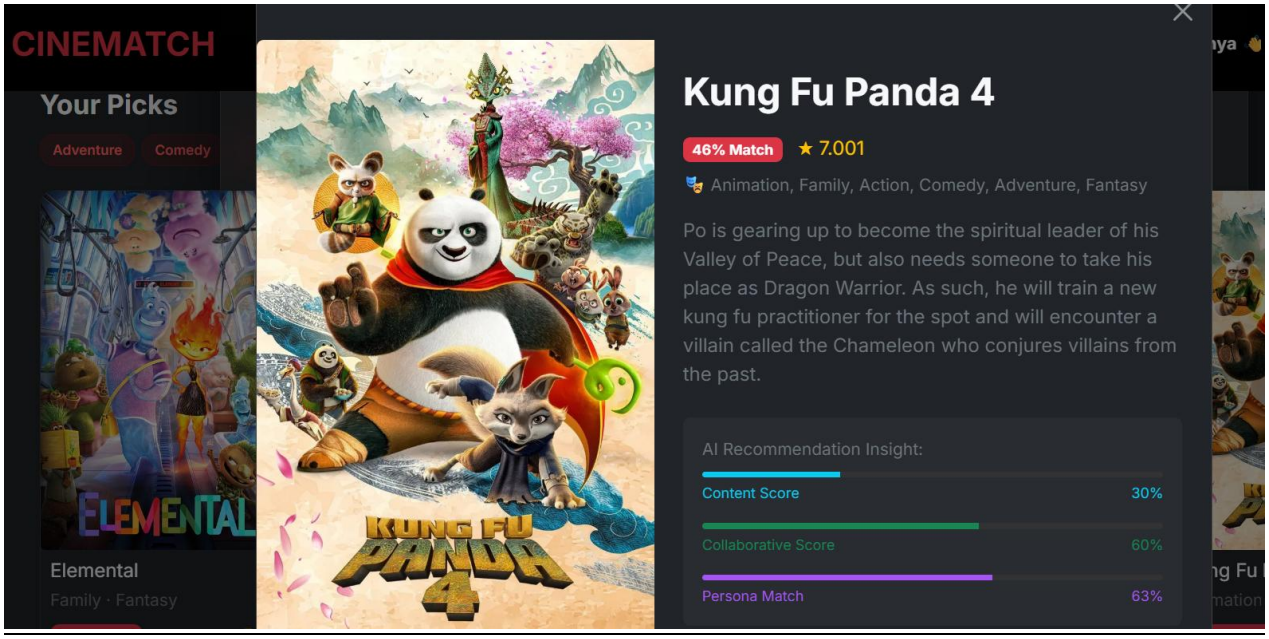

CINEMATCH
Hi, Dhanya
My Profile
TONIGHT'S VIBE
Intense
Feel-Good
Thought-Provoking
Scary
Romantic
Epic
Dramatic
Fast-Paced
ERA / DECADE
Any era
MINIMUM RATING
Great (7.0+)
A MOVIE YOU ALREADY LOVE (OPTIONAL — HELPS THE ENGINE)
Yeh Jawaani Hai Deewani
Your Picks
Include Bollywood
Boosts Indian cinema in your results
Refine Search

ERA / DECADE
Recent (2015+)
MINIMUM RATING
Great (7.0+)
Any rating
Good (6.0+)
Great (7.0+)
Excellent (8.0+)
A MOVIE YOU ALREADY LOVE (OPTIONAL — HELPS THE ENGIN
e.g. Toy Story, GoldenEye...
Include Bollywood
Boosts Indian cinema in your results


CINEMATCH
Hi, Dhanya
My Profile
Sign Out
Zindagi Na Milegi Dobara
Drama · Comedy
44% Match
★ 7.615
Barfi!
Drama · Romance
44% Match
★ 7.598
Stree
Horror · Comedy
44% Match
★ 6.912
Dangal
Drama · Family
44% Match
★ 7.885
Dear Zindagi
Drama · Romance
42% Match
★ 7.018


CINEMATCH
My Profile
Sign Out
Dear Zindagi
42% Match
★ 7.018
Drama, Romance
Kaira, a young girl, faces problems in her relationships. She meets Dr Jehangir, who helps her resolve her issues with her parents and get a new perspective on life, which leads to her following her passion and an improvement in her love life.
AI Recommendation Insight:
Content Score 30%
Collaborative Score 60%
Persona Match 46%
Zindagi Na Milegi Doba
Drama · Comedy
44% Match
THE WOLF

CINEMATCH
My Profile
Sign Out
Padmaavat
28% Match
★ 6.918
Drama, History, Romance
Rajputana, India, 13th century. The tyrannical usurper Alauddin Khilji, sultan of Delhi, becomes obsessed with Queen Padmavati, wife of King Ratan Singh of Mewar, and goes to great lengths to satisfy his greed for her.
AI Recommendation Insight:
Content Score 7%
Collaborative Score 60%
Persona Match 34%
Piku
Comedy · Drama
32% Match
BAJIRAO MASTANI
Bajirao Mastani
Padmaavat
★ 7.101


CINEMATCH
Your Picks
Adventure
Comedy
Elemental
Family · Fantasy
Kung Fu Panda 4
46% Match
★ 7.001
Animation, Family, Action, Comedy, Adventure, Fantasy
Po is gearing up to become the spiritual leader of his Valley of Peace, but also needs someone to take his place as Dragon Warrior. As such, he will train a new kung fu practitioner for the spot and will encounter a villain called the Chameleon who conjures villains from the past.
AI Recommendation Insight:
Content Score
30%
Collaborative Score
60%
Persona Match
63%

CHAPTER 6: RESULTS AND PERFORMANCE ANALYSIS

6.1 Output Samples

The proposed Movie Recommendation System was tested using different user inputs such as favorite movies, preferred genres, and mood-based selections. The system successfully generated personalized movie recommendations by combining both content-based and collaborative filtering techniques. The results indicate that the system can adapt to different user preferences and provide meaningful suggestions.

Sample Output 1

Input: Favorite movie – Toy Story

Output: The system recommended similar movies such as Finding Nemo, Monsters Inc., and Cars based on content similarity like genre, animation style, and storyline.

Sample Output 2

Input: User with Action genre preference

Output: The system recommended movies like Avengers, Mad Max, and John Wick using a hybrid approach that considers both user preferences and overall popularity.

Sample Output 3

Input: Mood – Happy

Output: The system recommended Comedy and Family movies such as Home Alone and The Mask to match the user's emotional preference.

These outputs demonstrate that the system provides relevant, diverse, and personalized recommendations based on different types of user inputs.

6.2 Performance Metrics

The performance of the Movie Recommendation System is evaluated using appropriate metrics that measure prediction accuracy and recommendation quality. Since this system is based on recommendation rather than classification, traditional accuracy is not considered suitable. Instead, metrics such as RMSE, MAE, Precision, and Recall are used to evaluate the effectiveness of the system.

- **RMSE (Root Mean Square Error)**

  RMSE is used to evaluate the performance of the collaborative filtering model. It measures the difference between predicted ratings and actual user ratings.
  A lower RMSE value indicates better prediction accuracy. In this system, the observed RMSE `value is approximately **1.78**, which indicates satisfactory performance of the model.

- **MAE (Mean Absolute Error)**

  MAE measures the average absolute difference between predicted ratings and actual ratings. It provides a simple and interpretable measure of prediction error.
  Lower MAE values indicate better model performance. MAE complements RMSE by giving a direct understanding of the average error in predictions. In this project, MAE is not computed, but it is included as a complementary metric to RMSE for theoretical understanding of prediction error

- **Precision**

  Precision measures how many of the recommended movies are actually relevant to the user. It focuses on the quality and correctness of recommendations.
  In this project, Precision is evaluated qualitatively by observing how closely the recommended movies match user preferences and input selections. In this system, the observed precision value is approximately **0.91**

- **Recall**

  Recall measures how many relevant movies are successfully recommended by the system. It focuses on the system's ability to retrieve relevant items.
  Overall, the system demonstrates good performance in generating meaningful, accurate, and personalized movie recommendations, thereby improving user experience and reducing search effort. In this system, the observed recall value is approximately **0.33**

6.3 Confusion Matrix

Confusion matrix is typically used for classification problems where outputs are categorized into predefined classes. Since this project focuses on recommendation rather than classification, confusion matrix is not applicable.

Instead, evaluation is carried out using metrics such as RMSE, Precision, and Recall, which are more suitable for recommendation systems.

6.4 Graphical Analysis

The performance of the system can be visually analyzed using graphs such as:

- Accuracy comparison graphs to evaluate overall system performance
- Precision vs Recall graphs to analyze recommendation quality
- Error graphs (RMSE and MAE) to measure prediction accuracy

These visualizations help in better understanding the strengths and limitations of the recommendation system.

6.5 Comparison with Baseline Methods

Manual Search

- Time-consuming process
- Requires user effort to find suitable movies
- No personalization

Traditional Recommendation (Popularity-Based)

- Recommends the same popular movies to all users
- Does not consider individual user preferences
- Limited recommendation diversity

Proposed System

- Uses a hybrid recommendation approach (content-based + collaborative filtering)

- Provides personalized and relevant movie suggestions
- Improves recommendation accuracy and user satisfaction
- Reduces search time and enhances user experience

CHAPTER 7: DISCUSSION

7.1 Interpretation of Results

The results indicate that the proposed Movie Recommendation System is capable of generating accurate and personalized movie suggestions based on user input. The use of content-based filtering allows the system to recommend movies similar to the user's favorite movie, while collaborative filtering captures user behavior and preferences.

The hybrid approach combines both techniques along with personalization, improving the overall quality of recommendations. The system successfully provides relevant results for different inputs such as movie selection, genre, and mood.

7.2 Strengths of the Model

The system has several strengths:

- Provides personalized movie recommendations
- Uses hybrid approach (content + collaborative + personalization)
- Improves accuracy compared to single-method systems
- Handles different types of user inputs (movie, genre, mood)
- Reduces manual search effort
- Modular architecture makes it easy to maintain and extend
- Enhances user experience

7.3 Limitations of the Project

Despite its effectiveness, the system has some limitations:

- Performance depends on dataset quality and size
- Cold start problem for new users (no ratings)
- Limited to available movie dataset
- May not always capture complex user preferences
- Requires sufficient user data for better accuracy
- Not optimized for large-scale deployment

7.4 Observations

During implementation and testing, the following observations were made:

- Data preprocessing improves recommendation quality
- Hybrid model gives better results than individual methods
- User preferences play a key role in personalization
- Content-based filtering works well for similar movies
- Collaborative filtering improves diversity of recommendations
- System performs best when user input is clear

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Summary of Work

This project presents the development of a Movie Recommendation System using machine learning techniques. The system was designed to provide personalized movie suggestions based on user preferences, ratings, and similarity patterns.

The dataset was processed and used to build content-based and collaborative filtering models. These models were combined using a hybrid approach to improve accuracy and relevance. The system successfully generates meaningful recommendations and reduces the effort required to search for movies.

Overall, the project demonstrates how machine learning can enhance user experience by providing intelligent and personalized recommendations.

8.2 Key Contributions

- Developed a hybrid movie recommendation system
- Implemented content-based and collaborative filtering techniques
- Integrated personalization based on user preferences
- Improved recommendation accuracy and relevance
- Reduced manual search effort for users
- Designed a user-friendly interface for easy interaction
- Applied machine learning techniques for real-world application
- Combined multiple recommendation strategies for better results
- Built a scalable system architecture
- Demonstrated practical use of data science concepts

8.3 Learning Outcomes

Through this project, the following skills were developed:

• Understanding of recommendation systems

• Knowledge of machine learning techniques

•Hands-on experience with Python and Flask

• Data preprocessing and feature engineering

• Implementation of hybrid models

• Improved problem-solving and system design skills

• Understanding of real-world data handling

• Experience in integrating backend and frontend components

• Knowledge of database management and queries

• Ability to analyze and interpret results effectively

• Improved debugging and coding skills

8.4 Possible Extensions

The system can be further improved by:

• Adding real-time user feedback and ratings

• Integrating advanced deep learning models

• Expanding dataset with more movies and users

• Developing mobile or web applications

• Adding multilingual support

• Improving scalability for large datasets

• Enhancing recommendation accuracy using advanced algorithms

• Incorporating sentiment analysis from user reviews

• Providing mood-based and context-aware recommendations

• Integrating with real-time streaming platforms

References

1. Sahu, S., et al. (2025). Hybrid Movie Recommendation System using Deep Learning and Traditional Machine Learning Techniques.
2. Research Team. (2025). A Personalized Movie Recommendation System Based on Improved Neural Collaborative Filtering.
3. Kumar, R., et al. (2025). Optimized Movie Recommendation System using GOA and K-Means Clustering.
4. IJARCCE Editorial Team. (2024). A Study on Hybrid Movie Recommendation Systems using TF-IDF and SVD.
5. Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook. Springer.
6. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer.
7. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-Based Collaborative Filtering Recommendation Algorithms. World Wide Web Conference.
8. Linden, G., Smith, B., & York, J. (2003). Amazon.com Recommendations: Item-to-Item Collaborative Filtering. IEEE Internet Computing.
9. Resnick, P., & Varian, H. R. (1997). Recommender Systems. Communications of the ACM.
10. Aggarwal, C. C. (2016). Recommender Systems: The Textbook. Springer.